

EcoTasks

Suivre ses actions écologiques au quotidien

Mini-Projet Numérique Durable

Partie 2 - Implémentation, Analyse et Évaluation

Rapport Livrable 2

2025-2026

Equipe : El Assad SAID, Yassine TENZEKHTI, Kelvish DUVAL, Lucas RINAUDO, Théotim DJELASSI-DOUBLET

Classe : I1AFR4

Professeur : GUIFO FODJO Yvan

Phase 1 - Cadrage du produit	3
1. Identifier le besoin et la proposition de valeur	3
2. Définir le MVP	4
3. Définir les utilisateurs et leurs rôles	4
4. Identifier les données manipulées	5
5. Rédiger des user stories	5
6. Construire un mini backlog	6
Phase 3 - Conception fonctionnelle	10
2. Diagrammes attendus	10
3. Conception de l'interface	12
4. Choix des technologies	14
Phase 4 - Implémentation - Front-end, Back-end et Base de données	15
1. Développement front-end	15
2. Développement back-end	15
3. Déploiement du site web	15
Phase 5 - Analyse de l'empreinte carbone	16
1. Outils utilisés	16
2. Protocole d'analyse	17
Phase 6 - Tests et validation	20
1. Tests fonctionnels	20
2. Tests de performance	21
3. Test de sécurité minimaux	24
Phase 8 - Discussion et conclusion	25
1. Défis rencontrés	25
2. Compromis entre fonctionnalités et sobriété	26
3. Pistes d'amélioration	26
4. Regard critique	26
Phase 9 - Annexe	27
1. Checklist avant remise	27
2. Ressources	27

Phase 1 - Cadrage du produit

1. Identifier le besoin et la proposition de valeur

1. **Quel besoin utilisateur votre site cherche-t-il à résoudre ?**

Aider les particuliers à passer à l'action sur l'écologie au quotidien en leur permettant de poser concrètement leurs gestes, suivre s'ils les tiennent, et visualiser leur impact.

2. **Quel type de site souhaitez-vous développer ?** Un mini questionnaire de tâches thématique sur la sensibilisation écologique.

3. **À qui s'adresse-t-il ?** Aux particuliers motivés mais désorganisés dans leurs engagements écologiques, et aux étudiants sensibilisés au numérique durable.

4. **Quelle valeur apporte-t-il aux utilisateurs ?** Un seul endroit pour planifier, valider et chiffrer ses actions écologiques (avec une estimation CO2 par geste, basée sur l'ADEME), dans une interface sobre et sans tracker.

5. **Pourquoi votre idée est-elle réaliste pour 4 à 5 étudiants ?** Le projet est minimal. Nous avons 2 entités (utilisateurs + tâches), un CRUD classique, une stack légère qu'on maîtrise (Flask + SQLite + HTML/CSS/JS), aucun service externe à intégrer, et un déploiement gratuit sur Render. Tout est faisable en quelques semaines sans compétences avancées.

2. Définir le MVP

1. **Quelle est la fonction centrale du site ?** Permettre à un utilisateur connecté de créer, suivre et cocher des tâches écologiques personnelles, avec une estimation CO2 par tâche.

2. **Quelles sont les fonctionnalités indispensables pour qu'il soit réellement utilisable ?** Inscription/connexion, CRUD complet sur les tâches (créer, lister, modifier, supprimer, cocher), un tableau de bord paginé avec filtre "à faire/accomplies", et la gestion du profil (modifier, supprimer son compte). Côté admin : lister les utilisateurs et pouvoir promouvoir/supprimer.

3. **Quelles fonctionnalités seraient intéressantes mais peuvent être reportées à une version ultérieure ?** Notifications, partage social, système de badges et classements, statistiques avancées en graphique, application mobile, intégration calendrier. Tout ça serait intéressant mais chaque ajout ne sert pas forcément le projet de base et aurait alourdi le site.

3. Définir les utilisateurs et leurs rôles

— **Qui peut consulter ?** Tout le monde.

— **Qui peut créer des données ?** Les utilisateurs connectés (leurs propres tâches).

— **Qui peut modifier ?** Chaque utilisateur sur ses propres tâches et son propre profil. L'admin peut en plus modifier le rôle d'un utilisateur.

— **Qui peut supprimer ?** Chacun ses tâches et son propre compte. L'admin peut supprimer n'importe quel compte utilisateur.

— **Qui administre l'application ?** Le rôle "administrateur" qui est un utilisateur connecté avec des droits supplémentaires.

4. Identifier les données manipulées

1. Quelles données liées aux utilisateurs seront stockées ?

Le strict nécessaire : id, nom, email, mot de passe (hashé bcrypt), rôle, date de création. Pas de date de naissance, pas d'adresse, pas de téléphone, pas de photo de profil, qui ne sont pas nécessaires dans ce que le site propose.

2. Quelle est votre seconde entité métier principale ?

La tâche écologique : id, titre, description, état accompli ou non, catégorie, impact CO2 estimé, l'utilisateur propriétaire, date de création.

3. Quelles relations existent entre les différentes données ?

Une relation 1-N car un utilisateur a plusieurs tâches, chaque tâche appartient à un seul utilisateur. La suppression d'un utilisateur supprime ses tâches mais la suppression d'une tâche ne supprime pas l'utilisateur.

4. Quelles informations sont réellement nécessaires, et lesquelles doivent être évitées pour rester sobres ?

Pas de tracking de localisation, pas de logs d'actions, pas de sessions historisées, pas de cookies tiers, pas d'avatar uploadé. Plus on stocke, plus on consomme, donc on ne stocke que ce qui sert au produit.

5. Rédiger des user stories

Gestion des utilisateurs :

- En tant que visiteur, je veux pouvoir m'inscrire avec un email et un mot de passe, afin de créer un compte personnel pour suivre mes actions écologiques.
- En tant qu'utilisateur connecté, je veux pouvoir modifier mon profil ou supprimer mon compte, afin de garder le contrôle sur mes données personnelles.

Notre entité métier :

- En tant qu'utilisateur connecté, je veux pouvoir créer une tâche écologique (Transport, Énergie, etc) afin de suivre mes engagements quotidiens.
- En tant qu'utilisateur connecté, je veux pouvoir filtrer mes tâches "à faire" et "accomplies" et naviguer dans une liste paginée, afin de retrouver rapidement ce qui m'importe sans charger toute ma base.

Sobriété du site :

- En tant qu'utilisateur, je veux que chaque tâche affiche une estimation d'impact CO2 économisé (basée sur l'ADEME), afin de comprendre la valeur écologique réelle de mes engagements sans avoir à consulter une source externe.

6. [Construire un mini backlog](#)

ID : 01

- **Fonctionnalité** : Inscription utilisateur (formulaire + bcrypt)
- **Priorité** : Haute
- **Difficulté** : Faible
- **Responsable** : Back-end

ID : 02

- **Fonctionnalité** : Connexion / déconnexion (sessions Flask)
- **Priorité** : Haute
- **Difficulté** : Faible
- **Responsable** : Back-end

ID : 03

- **Fonctionnalité** : CRUD tâches écologiques
- **Priorité** : Haute
- **Difficulté** : Moyenne
- **Responsable** : Back-end

ID : 04

- **Fonctionnalité** : Tableau de bord paginé + filtres
- **Priorité** : Haute
- **Difficulté** : Moyenne
- **Responsable** : Back-end / Front-end

ID : 05

- **Fonctionnalité** : Catégories + estimation CO2
- **Priorité** : Haute
- **Difficulté** : Faible
- **Responsable** : Back-end

ID : 06

- **Fonctionnalité** : Profil utilisateur (lecture / modification)
- **Priorité** : Moyenne
- **Difficulté** : Faible
- **Responsable** : Back-end

ID : 07

- **Fonctionnalité** : Suppression de compte avec confirmation
- **Priorité** : Moyenne
- **Difficulté** : Faible
- **Responsable** : Back-end/Front-end

ID : 08

- **Fonctionnalité** : Panel admin paginé (liste/promotion/suppression)
- **Priorité** : Moyenne
- **Difficulté** : Moyenne
- **Responsable** : Back-end

ID : 09

- **Fonctionnalité** : Mode sombre (préférence système + toggle manuel)
- **Priorité** : Basse
- **Difficulté** : Faible

- **Responsable** : Front-end

ID : 10

- **Fonctionnalité** : Optimisations Green IT (defer, index BDD, SELECT explicite)
- **Priorité** : Haute
- **Difficulté** : Faible
- **Responsable** : Tous

ID : 11

- **Fonctionnalité** : Déploiement Render + CI/CD GitHub
- **Priorité** : Haute
- **Difficulté** : Faible
- **Responsable** : DevOps

ID : 12

- **Fonctionnalité** : Tests fonctionnels documentés (≥ 8 scénarios)
- **Priorité** : Moyenne
- **Difficulté** : Faible
- **Responsable** : Tests

ID : 13

- **Fonctionnalité** : Mesures EcoIndex/Lighthouse/Website Carbon
 - **Priorité** : Haute
 - **Difficulté** : Faible
 - **Responsable** : Tous
-

Phase 3 - Conception fonctionnelle

2. Diagrammes attendus

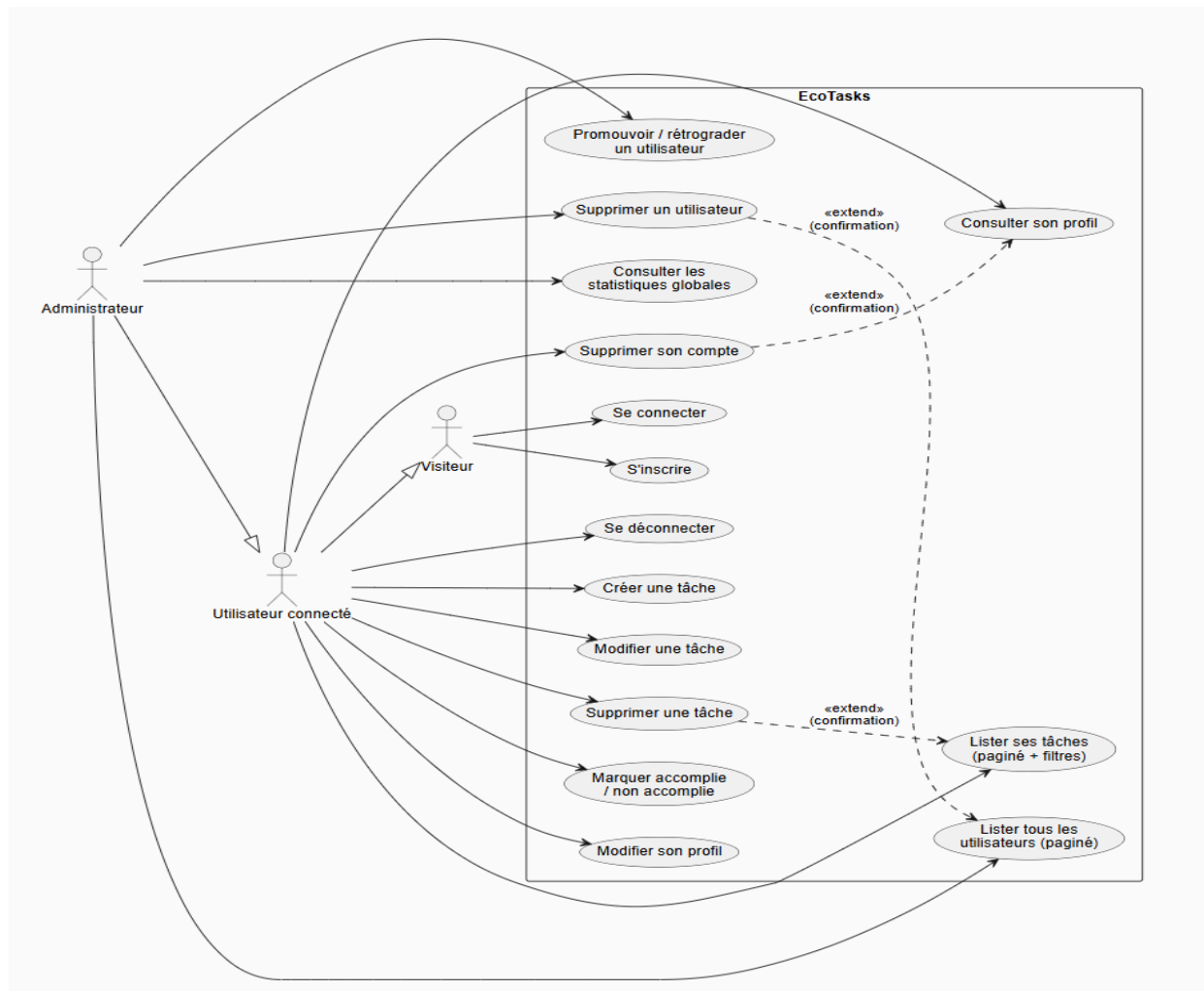


Diagramme de cas d'utilisation

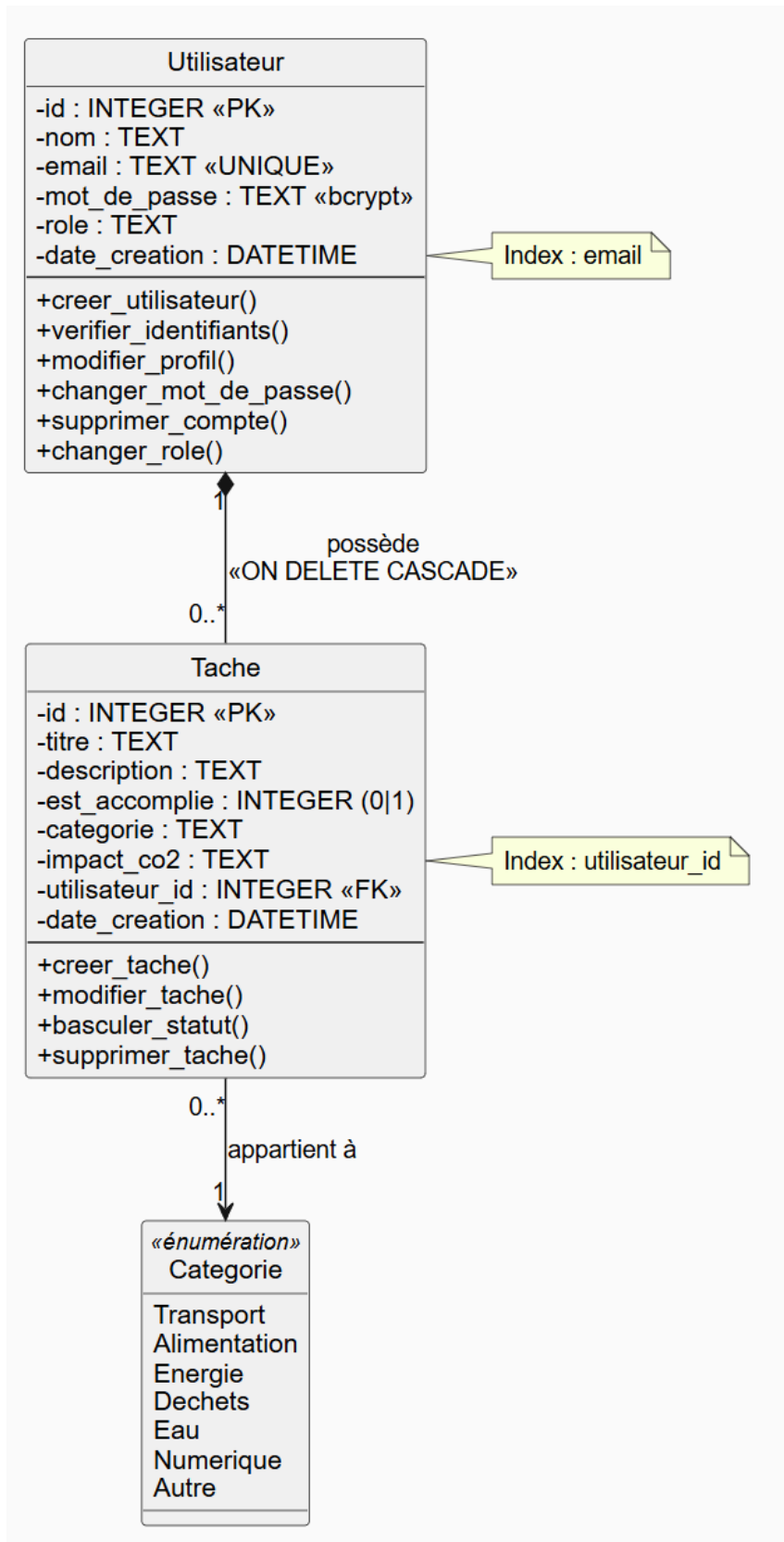


Diagramme de classe

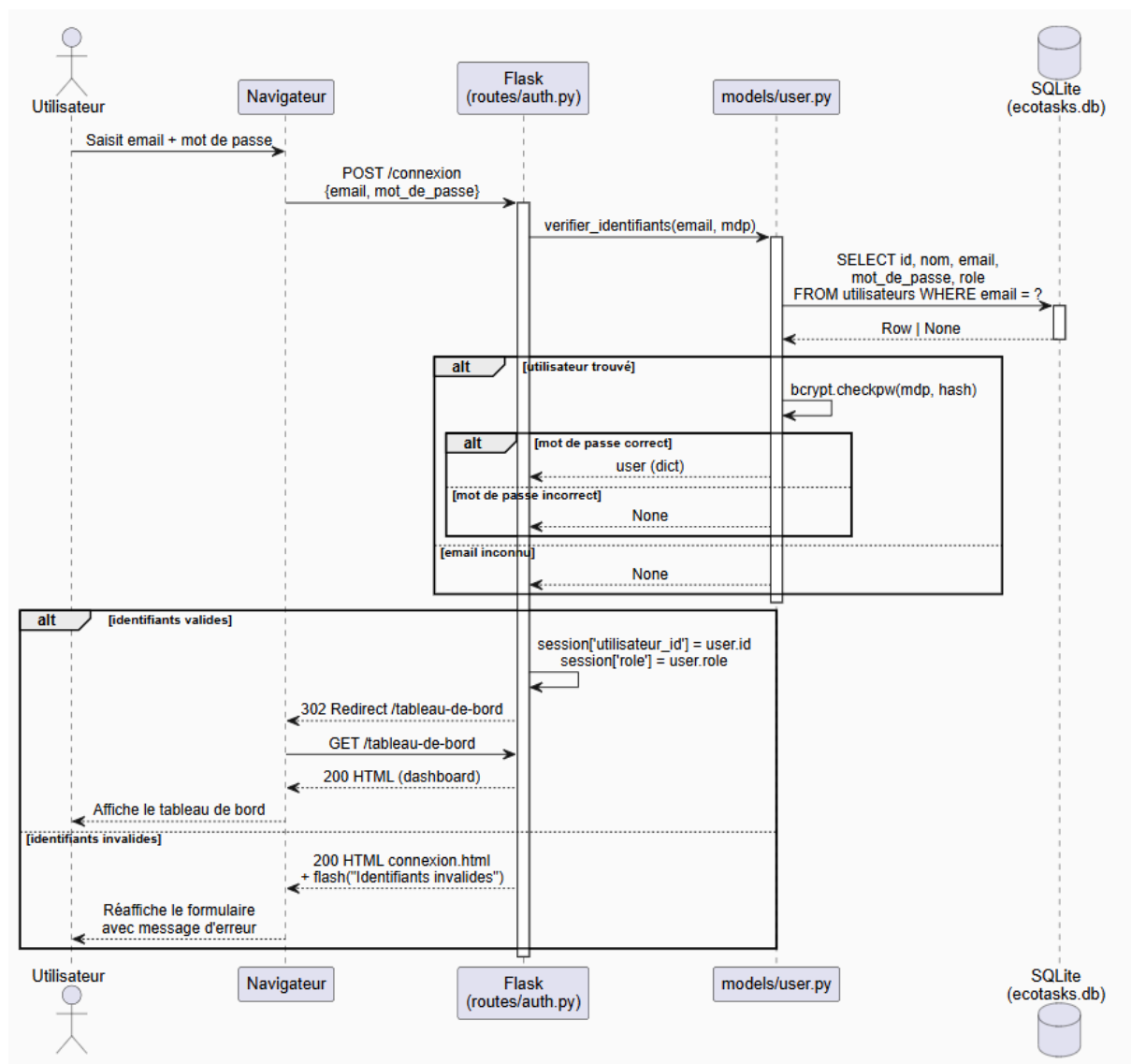
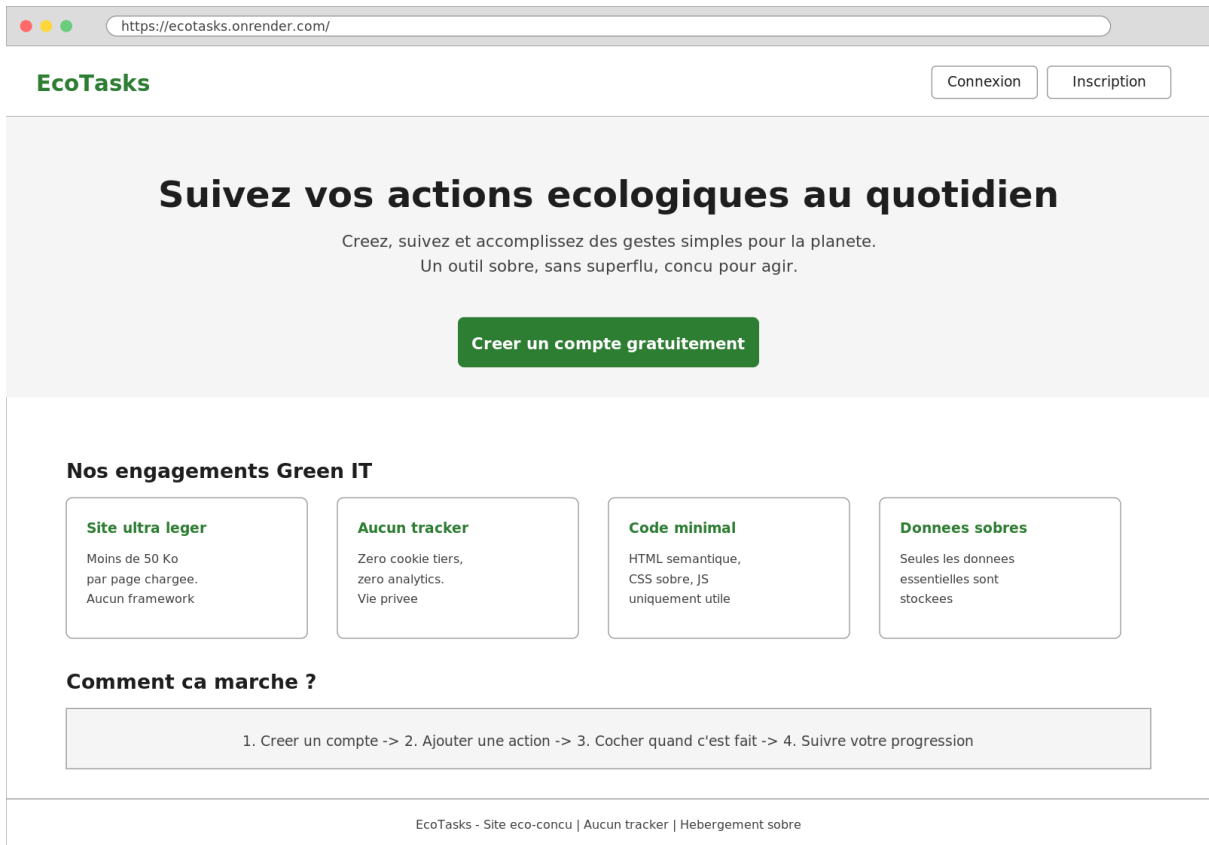


Diagramme de séquence

3. Conception de l'interface



Maquette - Page d'accueil

https://ecotasks.onrender.com/connexion

EcoTasks Connexion Inscription

Connexion

Accédez a vos actions ecologiques.

Adresse email
alice@exemple.fr

Mot de passe

Se connecter

[Pas encore de compte ?](#) [Créer un compte](#)

EcoTasks - Site eco-concu | Aucun tracker | Hébergement sobre

Maquette - Page de connexion

4. Choix des technologies

Front-end : HTML5 / CSS3 / JS - Aucun framework lourd (ex : React/Vue/Angular), chargement < 50 ko

Back-end : Python + Flask - Léger (~30 Mo), rapide à démarrer, 2 dépendances seulement (flask + bcrypt)

Base de données : SQLite - Fichier unique, pas de serveur supplémentaire, empreinte mémoire quasi nulle

Déploiement : Local - Pas de cloud lourd, démo en local suffisante pour le MVP

Documentation et suivi : GitHub (Issues, Projects, Pull Requests) -
Tout au même endroit que le code, pas d'outils tiers, maintenance facile,
traçabilité automatique grâce aux commits

Phase 4 - Implémentation - Front-end, Back-end et Base de données

1. Développement front-end

- Balises sémantiques (<header>, <nav>, <main>, <footer>, <details>).
- Script chargé avec defer, ne bloque pas le rendu initial.
- Polices système avec zéro requête externe
- Mode sombre
- Interface paginée (10 tâches/page) sans scroll infini
- Pas d'animations CSS sans valeur fonctionnelle.

2. Développement back-end

- routes/auth.py : inscription, connexion, déconnexion
- routes/tasks.py : CRUD tâches et dashboard paginé
- routes/users.py : profil utilisateur
- routes/admin.py : panel admin paginé
- models/user.py : accès BDD utilisateurs + bcrypt
- models/task.py : accès BDD tâches

3. Déploiement du site web

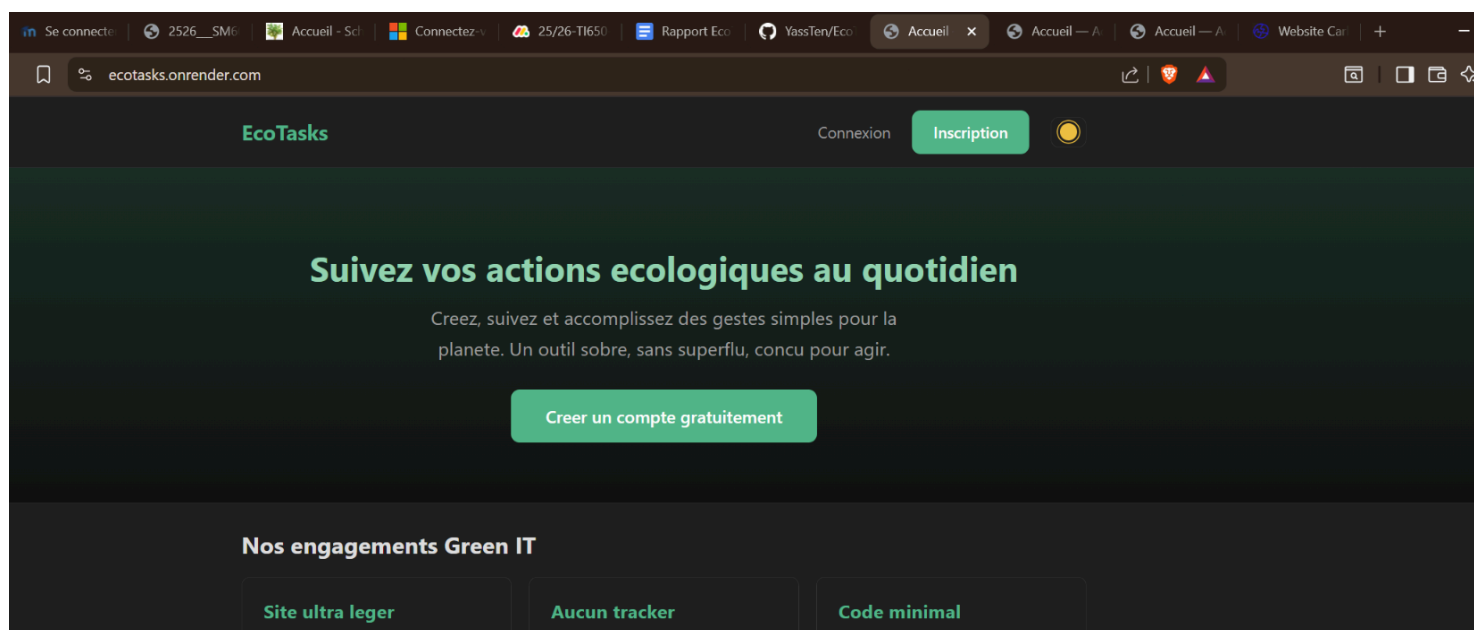
URL publique : <https://ecotasks.onrender.com>

Plateforme : [Render.com](https://render.com) car supporte Python

Serveur WSGI : gunicorn 23.0.0 car il possède une bonne sobriété et il est intégré avec Render

Configuration : Fichier "render.yaml". Render lit ce fichier pour configurer le site

Variables sensibles : CLE_SECRETE qui est générée automatiquement par Render lors de la création du service (generateValue: true), .env listé dans .gitignore, .env.example comme modèle.



Site déployé en ligne

Phase 5 - Analyse de l'empreinte carbone

1. Outils utilisés

1. Google Lighthouse : scores Performance, Accessibilité, Bonnes pratiques, SEO

2. EcoIndex (<https://www.ecoindex.fr>) : note A à G, empreinte CO2 par visite

3. Website Carbon Calculator (<https://www.websitecarbon.com>)
émissions CO2 par visite

2. Protocole d'analyse

1. Mesure initiale

Indicateur	Avant	Après	Gain
Poids de la page d'accueil	0 Ko	22 Ko	+22 Ko
Nombre de requêtes HTTP	0	1	+1
Score EcoIndex	96.61/100	95.95/100	-0.66pts
Note EcoIndex	A	A	-
CO2/visite	0.03g	0.03g	0 %
Score Lighthouse Perf.	100/100	100/100	0 pt
FCP	0.9s	0.9s	0%
LCP	0.9s	0.9s	0%
TBT	0ms	0ms	0
CLS	0	0	0
SI	0.9s	0.9s	0

Mesures de la page d'accueil

Indicateur	Avant	Après	Gain
Poids de la page de connexion	18 Ko	26 Ko	+8 Ko
Nombre de requêtes HTTP	3	3	0
Score EcoIndex	95.61/100	95.50/100	-0.11pts
Note EcoIndex	A	A	-
CO2/visite	0.03g	0.03g	0%
Score Lighthouse Perf.	100/100	100/100	0pt

Mesure de la page de connexion

Indicateur	Avant	Après	Gain
Poids de la page du tableau de bord	15 Ko	22 Ko	0%
Nombre de requêtes HTTP	4	4	0
Score EcoIndex	83.11/100	90.99/100	+7.88pts
Note EcoIndex	A	A	-
CO2/visite	-	-	-
Score Lighthouse Perf.	100/100	100/100	0pt
FCP	1.1s	0.9s	-0.2s
LCP	1.1s	1.1s	0
TBT	0ms	0ms	0

CLS	0	0	0
SI	1.1s	0.9s	-0.2s

La mesure de la page du tableau de bord s'est fait après la création de 30 tâches afin d'avoir une estimation plus concrète

La valeur des métriques (FCP, LCP, TBT, CLS, SI) sont les mêmes pour la page d'accueil et de connexion (avant et après)

2. Identification des sources de pollution numérique

- SELECT * utilisé dans l'ancienne version => corrigée dans la nouvelle
- Absence de pagination => corrigée dans la nouvelle version avec LIMIT/OFFSET
- Absence d'index BDD => corrigée avec deux index

3. Mise en oeuvre des optimisations

Nous avons appliqués **6 optimisations** entre l'ancienne et la version finale :

1. **Polices système** : remplacement des polices web par "system-ui" => aucune requête externe, environ 50 Ko économisés par page
2. **Pagination côté serveur** : LIMIT/OFFSET avec 10 résultats par page sur le tableau de bord; le DOM ne croît plus avec le nombre de tâches
3. **Attribut defer sur le script** : le rendu n'est plus bloqué par le JavaScript; gain sur le First Contentful Paint
4. **SELECT explicites** : récupération uniquement des colonnes utiles côtés BDD et moins de données entre SQLite et Flask
5. **Index BDD** : index sur taches.utilisateur_id et utilisateurs.email ainsi les requêtes WHERE sont accélérées

6. Aucun framework au niveau du front

Phase 6 - Tests et validation

1. Tests fonctionnels

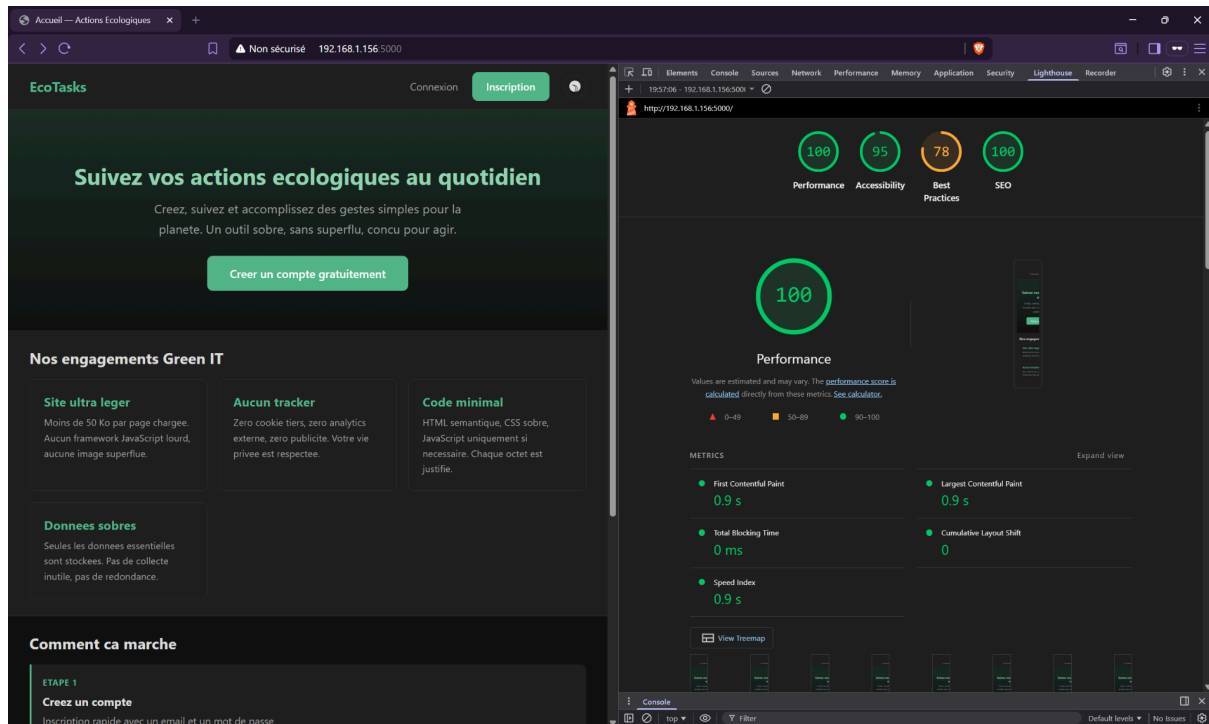
Scénario	Résultat attendu	Statut
Inscription valide	Utilisateur enregistré en BDD, redirection	Ok
Inscription email vide	Message d'erreur	Ok
Inscription email déjà pris	Message "compte existe déjà"	Ok
Inscription MDP < 6 car.	Message d'erreur	Ok
Modifier profil	Données mises à jour en BDD	Ok
Supprimer compte	Utilisateur supprimé après confirmation	Ok
Lister utilisateurs (admin)	Affichage paginé correct	Non testé
Créer une tâche	Tâche enregistrée, liste mise à jour	Ok
Tâche titre vide	Message d'erreur	Ok
Modifier une tâche	Liste mise à jour	Ok
Supprimer une tâche	Liste mise à jour	Ok
Connexion valide	Redirection vers espace personnel	Ok

Connexion avec mauvais MDP

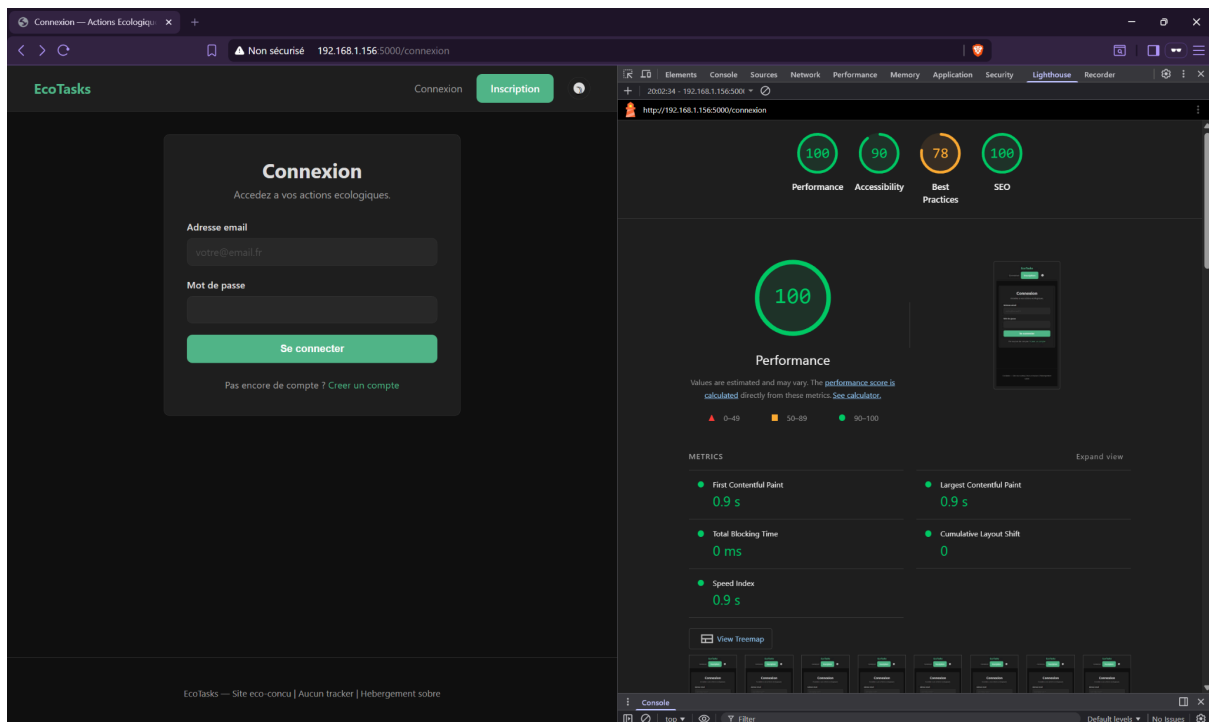
Message d'erreur, pas de session créée

Ok

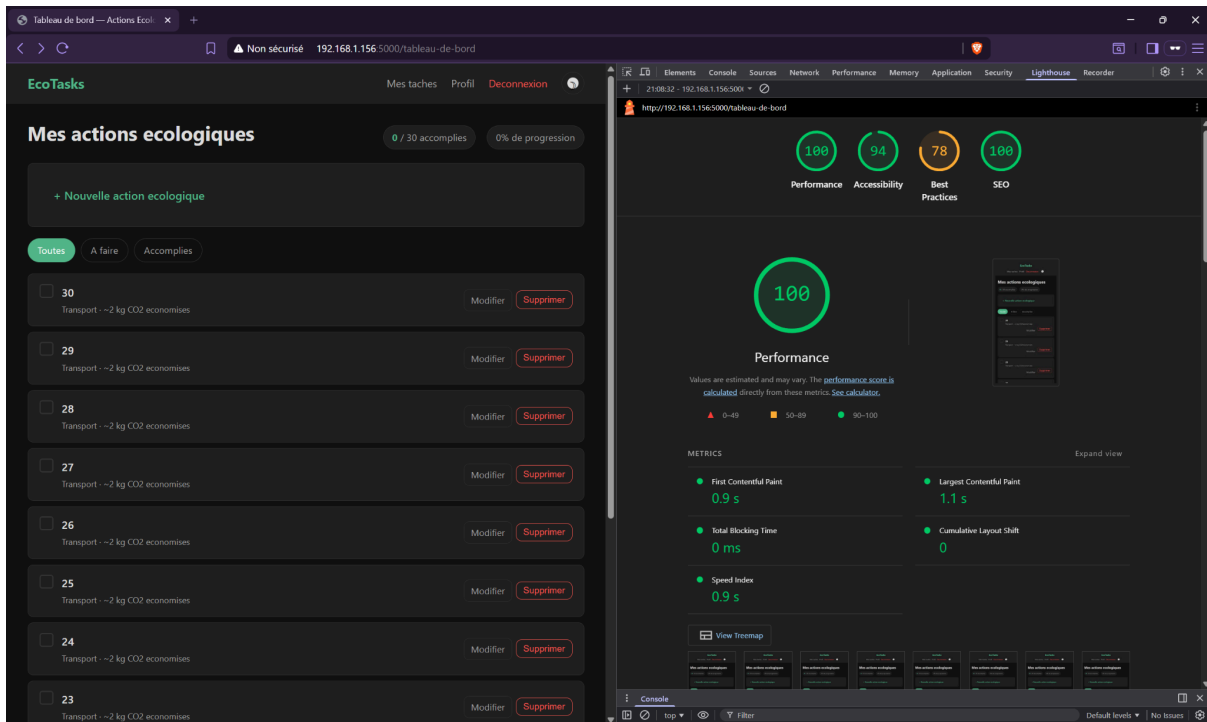
2. Tests de performance



Lighthouse - Page d'accueil



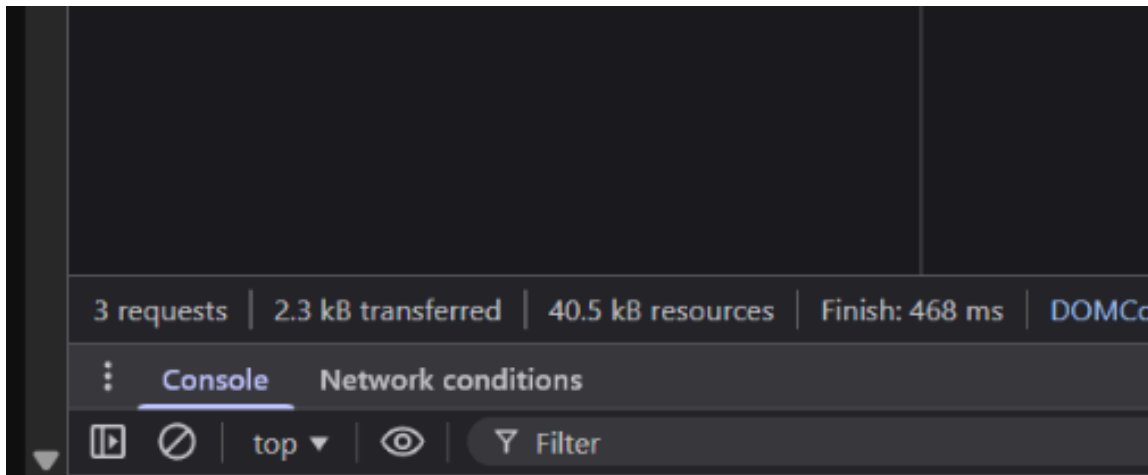
Lighthouse - Page de connexion



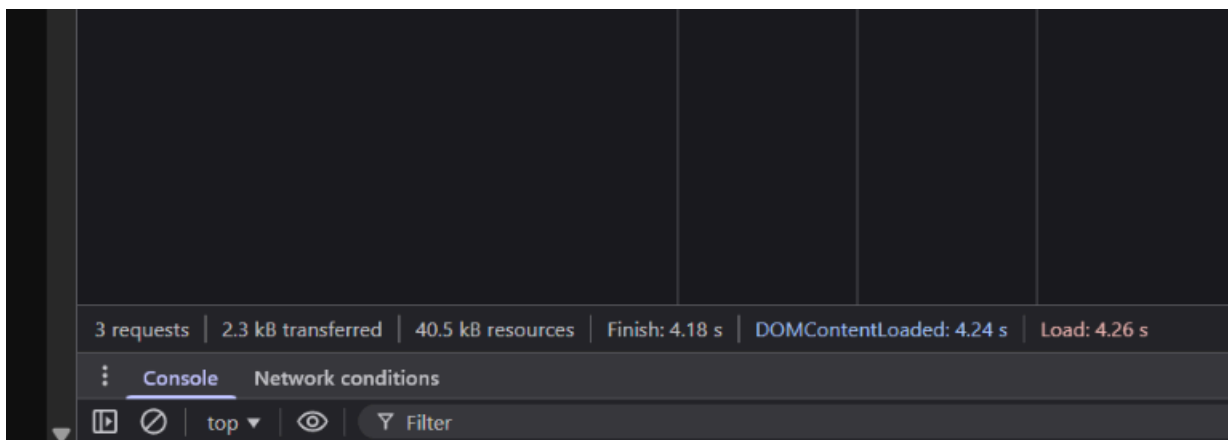
Lighthouse - Page du tableau de bord

Le Score Best Practices est de 78 car l'IP réseau privée utilisée n'est pas considérée comme sécurisée par Chrome. Ce n'est donc pas une régression technique.

- Réseau simulé 3G



Page du tableau de bord (30 tâches) - Test réseau Fast 4G



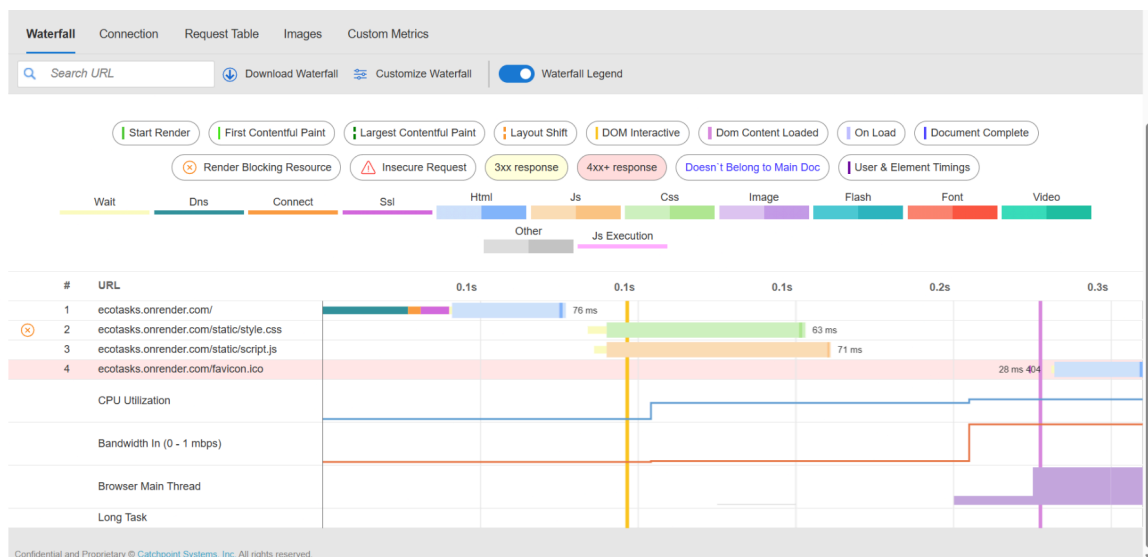
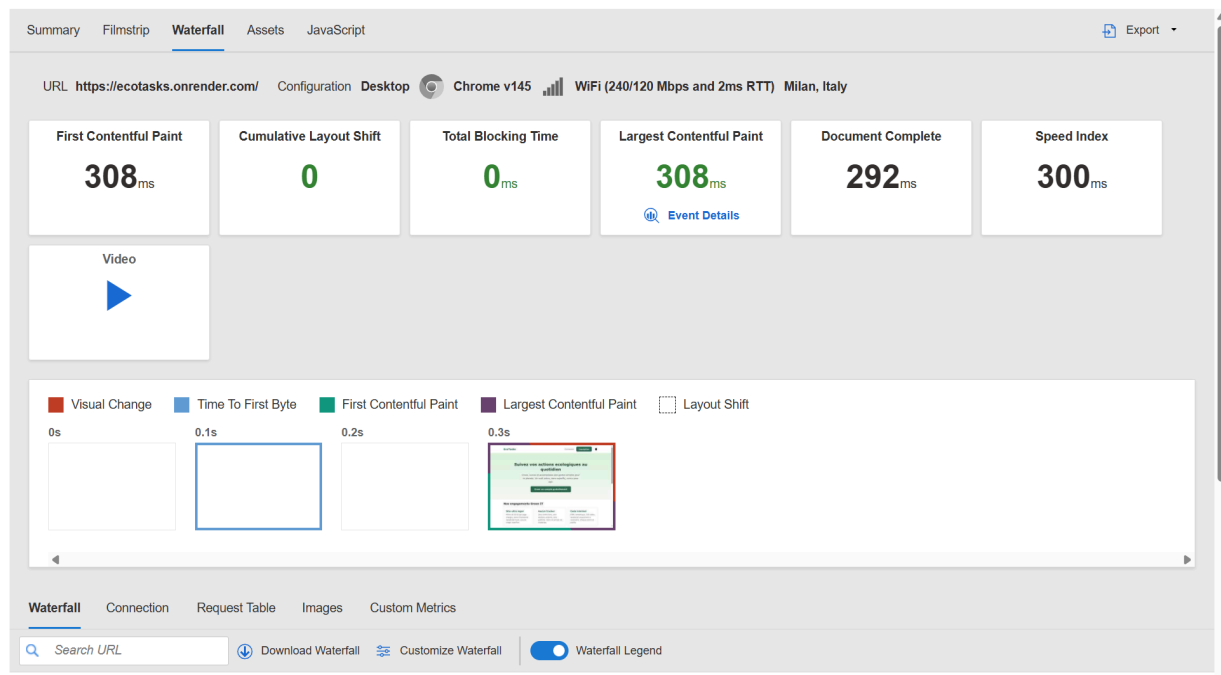
Page du tableau de bord (30 tâches) - Test réseau 3G

Nous pouvons observer que nous passons d'un temps de chargement de 468 ms à 4.18 s en passant à un réseau 3G. Le réseau 3G est donc environ 9 fois plus long.

- Test WebPageTest

URL:

<https://www.catchpoint.com/webpagetest/results?publicurl=https%3A%2F%2Fpublic.catchpoint.com%2FUI%2FEntry%2FWPTITP%2FARUJ-D-E-B2ACNEIjeqKbrXYAA-N>



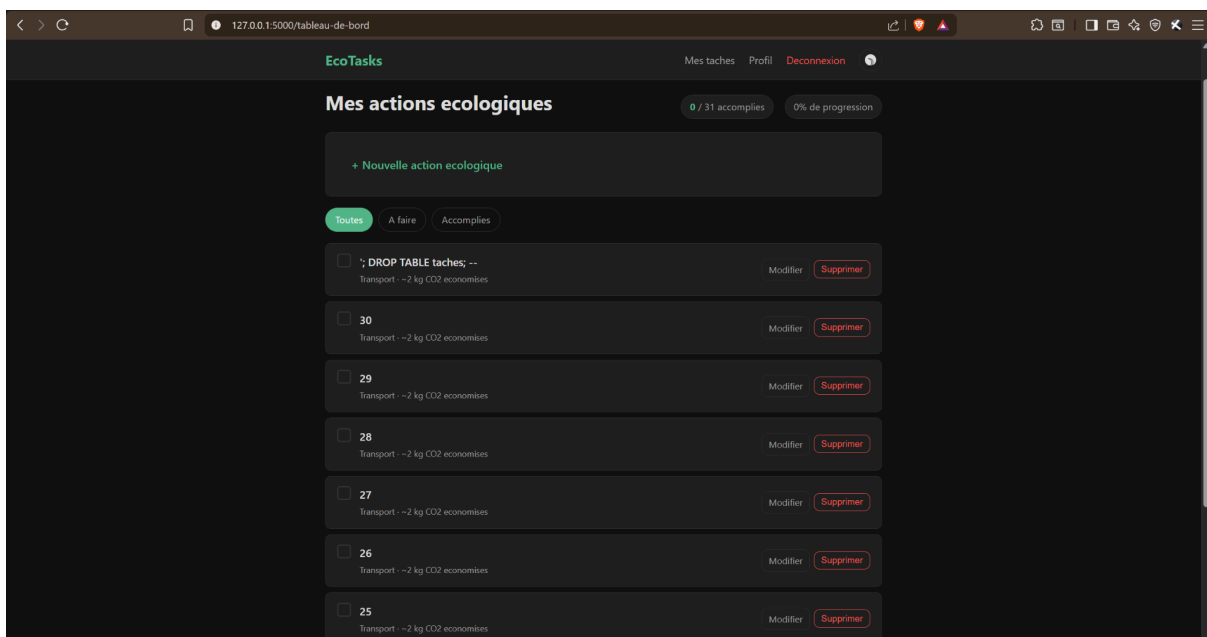
Waterfall de chargement de la Page d'accueil

Nous remarquons que la ressource bloquante ici est :
 "ecotasks.onrender.com/static/style.css"

3. Test de sécurité minimaux

Vérification	Méthode	Résultat
--------------	---------	----------

Aucun MDP en clair en BDD	SELECT mot_de_passe FROM utilisateurs LIMIT 1;	Hash bcrypt \$2b\$12\$.. - Ok
Aucune variable sensible dans Git	git log --all -p Select-String -Pattern "password= secret= a pi_key=" -CaseSensitive:\$false	Aucun résultat - Ok
Formulaires anti-injection SQL	'; DROP TABLE taches; --	Injection ignorée - Ok
Pages protégées inaccessibles sans session	Navigation directe vers /tableau-de-bord en navigation privée	Redirection /connexion - Ok



Test Injection SQL

```

PS C:\Users\sea10\OneDrive\Documents\Cours\I1\S6\Numérique Durable\ND-project-final\EcoTasks> python -c "import sqlite3; c = sqlite3.connect('ecotasks.db'); print(c.execute('SELECT
id, email, substr(mot_de_passe,1,20) FROM utilisateurs LIMIT 3').fetchall())"
[(1, 'test01@gmail.com', '$2b$12$FKcsrLVF8rvuo')]
PS C:\Users\sea10\OneDrive\Documents\Cours\I1\S6\Numérique Durable\ND-project-final\EcoTasks>

```

Test bcrypt hash BDD

Phase 8 - Discussion et conclusion

1. Défis rencontrés

1. **SQLite et foreign keys** : désactivées par défaut. PRAGMA foreign_keys = ON ajouté à chaque connexion dans get_db(), sinon ON DELETE CASCADE ne fonctionnait pas et la suppression d'utilisateur laissait des tâches orphelines
2. **Sessions Flask en production** : la secret_key doit être stable entre redémarrages => définie via la variable d'environnement CLE_SECRETE sur Render
3. **Pagination + filtres combinés** : conserver le filtre actif dans les liens de pagination => résolu en passant filtre=filtre_actif dans url_for('taches.tableau_de_bord',...)

2. Compromis entre fonctionnalités et sobriété

- **Stats calculées côté serveur** : pas de requête AJAX, Jinja2 injecte directement les statistiques. Évite un aller-retour réseau supplémentaire
- **Pas de WebSocket** : une page sobre rechargée suffit
- **Pas de CDN externe** : tous les assets sont servis par Flask, ce qui évite une dépendance réseau externe et un risque de fuite vers un tiers

3. Pistes d'amélioration

- Minification CSS / JS en production (cssnano, terser) => gain attendu d'environ 30 %

- Internationalisation FR / EN si l'application doit dépasser le cadre étudiant

4. Regard critique

SQLite est un bon choix pour le MVP, mais il deviendrait limitant si le projet grandissait (un seul écrivain à la fois, mal adapté à plusieurs serveurs en parallèle). L'absence de tests automatisés est une dette technique à combler avant d'aller plus loin. Finalement, l'expérience confirme une chose : la sobriété numérique est bien plus efficace quand elle guide la conception dès le départ que quand on essaie de l'ajouter à un site déjà lourd.

Phase 9 - Annexe

1. Checklist avant remise

- Site accessible via URL publique dans README.md
- Aucune variable sensible dans le code versionné
- Rapport PDF dans /docs/
- Tableau avant/après empreinte carbone complété avec valeurs réelles
- Captures Lighthouse jointes pour 3 pages
- Base de données initialisée (script SQL fourni)

2. Ressources

- **GR491 Green IT** : <https://gr491.isit-europe.org>
- **ADEME, estimations CO2** : <https://agirpoulatransition.ademe.fr>
- **EcoIndex** : <https://www.ecoindex.fr>
- **Website Carbon** : <https://www.websitecarbon.com>
- **PageSpeed Insights** : <https://pagespeed.web.dev>
- **The Green Web Foundation** :
<https://www.thegreenwebfoundation.org>